Cartpole 关键代码导读

这份 README 不重复运行步骤，重点展示当前 `cartpole` 闭环实验里最关键的代码，并说明这些代码在实验环中的作用。

相关文件：

- cartpole_setup.py
- maxone_with_filter.cpp
- cartpole_task.cpp
- training_controller.cpp

1. Python 侧：生成 stimulation sequence 和 runtime config

`cartpole_setup.py` 负责把 MaxOne/MaxLab 需要的刺激序列和运行参数准备好，然后启动 C++ 闭环进程。

1.1 encoding stimulation sequence

下面这段代码只生成两路基础脉冲序列：`encode_left_pulse` 和 `encode_right_pulse`。
C++ 侧不会每次实时创建新序列，而是按频率重复触发这两个已经持久化的 sequence。

```
def prepare_encoding_sequences(unit_ids: Sequence[int], all_unit_ids:
Sequence[int]) -> Dict[str, mx.Sequence]:
    if len(unit_ids) != 2:
        raise RuntimeError("Expected exactly two encoding stimulation units")
    sequences: Dict[str, mx.Sequence] = {}
    names = ["encode_left_pulse", "encode_right_pulse"]
    for name, unit_id in zip(names, unit_ids):
        seq = mx.Sequence(name=name, persistent=True)
        append_pulse_for_unit(seq, unit_id, all_unit_ids, f"{name}_pulse")
        seq.send()
        sequences[name] = seq
    return sequences
```

作用：

- 预生成两路 `encoding` 脉冲
- 后续由 C++ 根据 `theta` 对应的频率来触发
- 避免在实时闭环里动态拼接刺激序列

1.2 adaptive training sequence

论文里的训练信号是从 `training electrodes` 里取两两组合，形成 `paired-pulse pattern`。这里用 `train_pair_i_j` 的方式全部预生成。

```
def prepare_training_sequences(training_unit_ids: Sequence[int], all_unit_ids:
Sequence[int]) -> List[str]:
```

```
    if len(training_unit_ids) < 2:
        raise RuntimeError("At least two training units are required")
    pattern_names: List[str] = []
    inter_pulse_samples = int(STIM_PARAMS["inter_pulse_interval_ms"] * 1000 / 50)
    period_samples = int(20000 / STIM_PARAMS["training_frequency_hz"])
    repetitions = int((TRAINING_WINDOW_MS / 1000.0) *
STIM_PARAMS["training_frequency_hz"])

    for first_idx, second_idx in
itertools.combinations(range(len(training_unit_ids)), 2):
        name = f"train_pair_{first_idx}_{second_idx}"
        seq = mx.Sequence(name=name, persistent=True)
        first_unit = training_unit_ids[first_idx]
        second_unit = training_unit_ids[second_idx]
        for repetition in range(repetitions):
            append_pulse_for_unit(seq, first_unit, all_unit_ids, f"
{name}_a_{repetition}")
            seq.append(mx.DelaySamples(inter_pulse_samples))
            append_pulse_for_unit(seq, second_unit, all_unit_ids, f"
{name}_b_{repetition}")
            seq.append(mx.DelaySamples(max(0, period_samples -
inter_pulse_samples)))
        seq.send()
        pattern_names.append(name)
    return pattern_names
```

作用：

- 预生成所有 `adaptive training` 可选 pattern
- 每个 pattern 是一个固定的 `paired-pulse` 序列
- C++ 侧只需要在 episode 结束后发送被选中的 `sequence_name`

1.3 导出 runtime config

这一步把论文关键参数导出为 JSON，供 C++ 侧读取。

```
runtime_config = {
    "target_well": target_well,
    "read_window_ms": READ_WINDOW_MS,
    "training_window_ms": TRAINING_WINDOW_MS,
    "show_gui": bool(show_gui),
    "wait_for_sync": True,
    "channel_count": 1024,
    "experiment_duration_s": duration_minutes * 60,
    "cycle_duration_s": CYCLE_DURATION_S,
    "rest_duration_s": 0 if mode == "continuous_adaptive" else REST_DURATION_S,
    "encoding_scale_a": 7.0,
    "encoding_scale_b": 0.15,
    "ema_alpha": 0.2,
    "force_scale_n": 10.0,
    "sample_rate_hz": 20000.0,
```

```
    "threshold_multiplier": 3.0,
    "min_threshold": -20.0,
    "refractory_samples": 1000,
    "decoding_left_channels": list(decoding_left_channels),
    "decoding_right_channels": list(decoding_right_channels),
    "encoding_left_sequence": "encode_left_pulse",
    "encoding_right_sequence": "encode_right_pulse",
    "training_pattern_names": list(training_pattern_names),
    "log_path": str(log_path),
    "random_seed": 12345,
    "mode": mode,
}
```

作用：

- 把 `read window`、`training window`、`EMA alpha`、`force scale` 等关键参数固定下来
- 指定 decoding 用哪些 channels
- 指定 encoding 和 training 使用哪些 sequence 名称

2. C++ 主闭环：一次 read window 内做什么

闭环核心在 maxone_with_filter.cpp。
每个 `read_window_ms` 结束后，会做一次完整的“读 spikes -> 解码 -> 更新环境 -> 编码刺激 -> 判断是否训练”。

2.1 读 spike、做 EMA、计算力

```
const double left_count = sumCounts(state.spike_counts,
config.decoding_left_channels);
const double right_count = sumCounts(state.spike_counts,
config.decoding_right_channels);
state.left_rate = config.ema_alpha * state.left_rate + (1.0 - config.ema_alpha) *
left_count;
state.right_rate = config.ema_alpha * state.right_rate + (1.0 - config.ema_alpha)
* right_count;

const double force_newtons = clampUnitForce(config.force_scale_n, state.left_rate,
state.right_rate);
const bool terminal = state.task.step(force_newtons);
```

作用：

- 从左右两组 `decoding channels` 统计 spike count
- 用 `EMA` 平滑为左右 firing rate
- 用 `left_rate - right_rate` 计算动作力
- 把力传给 `CartpoleTask::step()`

这里的力经过 `clampUnitForce()` 处理：

```
double clampUnitForce(double force_scale_n, double left_rate, double right_rate) {
    const double unit_force = std::clamp(left_rate - right_rate, -1.0, 1.0);
    return unit_force * force_scale_n;
}
```

也就是先裁剪到 `[-1, 1]`，再乘 `10N`。

2.2 根据 pole angle 计算 encoding frequency

```
const double theta = state.task.getPoleAngleRad();
const double frequency_left =
    config.encoding_scale_a * std::pow(config.encoding_scale_b - std::sin(theta),
2.0);
const double frequency_right =
    config.encoding_scale_a * std::pow(config.encoding_scale_b + std::sin(theta),
2.0);

emitRatePulse(config.encoding_left_sequence, frequency_left,
config.timestep_seconds, &state.left_phase);
emitRatePulse(config.encoding_right_sequence, frequency_right,
config.timestep_seconds, &state.right_phase);
```

作用：

- 从当前 `theta` 计算论文里的双输入频率编码
- 左右两个 `encoding sequence` 以不同频率触发
- `emitRatePulse()` 用 phase accumulator 的方式把连续频率离散成 sequence 发送事件

`emitRatePulse()` 的核心逻辑如下：

```
void emitRatePulse(const std::string& sequence_name, double frequency_hz, double
dt_seconds, double* phase) {
    if (sequence_name.empty() || phase == nullptr) return;
    *phase += frequency_hz * dt_seconds;
    while (*phase >= 1.0) {
        maxlab::verifyStatus(maxlab::sendSequence(sequence_name.c_str()));
        *phase -= 1.0;
    }
}
```

这个实现的意义是：

- 配置层只需要准备一个基础 pulse
- 运行时通过累计相位来近似任意连续频率
- 比“预生成很多离散频率 sequence”更轻量

2.3 episode 终止后触发 adaptive training

```
if (terminal) {
    const double reward_seconds = state.task.getTimeBalanced();
    const TrainingDecision decision = state.trainer.onEpisodeEnd(reward_seconds);
    ++state.episode_index;
    state.logger.writeEpisode(
        state.episode_index,
        reward_seconds,
        decision.mean_5,
        decision.mean_20,
        decision.delivered,
        decision.sequence_name,
        theta);

    if (decision.delivered) {

maxlab::verifyStatus(maxlab::sendSequence(decision.sequence_name.c_str()));
        state.training_until = now +
std::chrono::milliseconds(config.training_window_ms);
    }

    resetEpisodeState(state);
}
```

作用：

- `reward` 直接使用本次 episode 的 `time balanced`
- 交给 `TrainingController` 判断是否训练、训练哪一个 pattern
- 如果需要训练，就发送对应的 `train_pair_i_j`
- 进入 `training_window_ms`，这段时间不再继续常规 encoding

3. Cartpole 环境本身：状态如何更新

`CartpoleTask` 是一个独立的任务状态机，专门负责 cartpole 物理状态，不直接处理硬件。

3.1 初始化参数

```
CartpoleTask::CartpoleTask(double timestep_seconds)
    : timestep_seconds_(timestep_seconds),
      gravity_(9.8),
      masscart_(1.0),
      masspole_(0.1),
      total_mass_(masscart_ + masspole_),
      length_(0.5),
      polemass_length_(masspole_ * length_),
      theta_threshold_radians_(16.0 * kPi / 180.0) {
    reset();
}
```

作用：

- 设定 cartpole 动力学参数
- 终止条件固定为 `|theta| > 16°`
- timestep 默认和 `read_window_ms` 对齐

3.2 单步环境更新

```
bool CartpoleTask::step(double force_newtons) {
    last_force_newtons_ = force_newtons;

    const double costheta = std::cos(theta_);
    const double sintheta = std::sin(theta_);
    const double temp = (force_newtons + polemass_length_ * theta_dot_ *
theta_dot_ * sintheta) / total_mass_;
    const double thetaacc =
        (gravity_ * sintheta - costheta * temp) /
        (length_ * (4.0 / 3.0 - masspole_ * costheta * costheta / total_mass_));
    const double xacc = temp - polemass_length_ * thetaacc * costheta /
total_mass_;

    x_ += timestep_seconds_ * x_dot_;
    x_dot_ += timestep_seconds_ * xacc;
    theta_ += timestep_seconds_ * theta_dot_;
    theta_dot_ += timestep_seconds_ * thetaacc;
    time_balanced_seconds_ += timestep_seconds_;

    return isTerminal();
}
```

作用：

- 根据当前力和当前状态推进一步 cartpole dynamics
- 更新时间累计值 `time_balanced_seconds_`
- 返回是否 terminal

这保证了环境更新逻辑与 Maxwell 硬件层解耦，后续你如果要换任务，可以保留闭环框架，只替换任务类。

4. Adaptive training：训练策略是怎么实现的

`TrainingController` 管理最近 episode 的 reward、pattern value、eligibility trace 和采样决策。

4.1 训练触发条件

```
TrainingDecision TrainingController::onEpisodeEnd(double reward_seconds) {
    updateValues(reward_seconds);

    recent_rewards_.push_back(reward_seconds);
    if (recent_rewards_.size() > 20) {
        recent_rewards_.pop_front();
```

```
    }

    TrainingDecision decision;
    decision.mean_5 = movingAverage(5);
    decision.mean_20 = movingAverage(20);

    decayEligibility();

    if (pattern_names_.empty()) {
        return decision;
    }

    if (recent_rewards_.size() < 5) {
        return decision;
    }

    if (decision.mean_5 > decision.mean_20) {
        return decision;
    }
```

作用：

- 每个 episode 结束时更新 reward history
- 计算最近 `5` 个 episode 和最近 `20` 个 episode 的 moving average
- 只有在 `mean_5 <= mean_20` 时才允许训练

4.2 pattern value 更新

```
void TrainingController::updateValues(double reward_seconds) {
    for (std::size_t i = 0; i < values_.size(); ++i) {
        values_[i] += value_alpha_ * (reward_seconds - values_[i]) *
eligibility_[i];
        values_[i] = (std::max)(values_[i], min_reward_);
    }
}

void TrainingController::decayEligibility() {
    for (double& value : eligibility_) {
        value *= eligibility_gamma_;
    }
}
```

作用：

- 用本次 `reward_seconds` 更新每个 pattern 的 value
- `eligibility trace` 会逐步衰减
- `value` 不允许低于 `min_reward`

4.3 按 value 加权采样 pattern

```cpp
int TrainingController::samplePatternIndex() {
    const double total = std::accumulate(values_.begin(), values_.end(), 0.0);
    if (total <= 0.0) {
        return -1;
    }

    std::uniform_real_distribution<double> dist(0.0, total);
    double draw = dist(rng_);
    for (std::size_t i = 0; i < values_.size(); ++i) {
        draw -= values_[i];
        if (draw <= 0.0) {
            return static_cast<int>(i);
        }
    }
    return static_cast<int>(values_.size() - 1);
}
```

作用：

- `value` 越高的 pattern，被选中的概率越大
- 当某个 pattern 在历史上更有利于更长的 episode 时，它更容易被再次训练

5. 你阅读代码时最值得先看的顺序

建议按这个顺序读：

1. cartpole_setup.py
2. maxone_with_filter.cpp
3. cartpole_task.cpp
4. training_controller.cpp

原因：

- 先看 Python，能知道实验到底给 C++ 传了什么配置
- 再看 C++ 主循环，能看到完整闭环链路
- 最后看 task 和 trainer，分别理解环境动力学和训练决策

6. 当前实现里最关键的工程分层

现在这套 `cartpole` 代码可以按四层来理解：

- `cartpole_setup.py`：实验准备层
- `maxone_with_filter.cpp`：实时闭环控制层
- `CartpoleTask`：任务环境层
- `TrainingController`：训练策略层

这四层分开之后，后面如果你要继续做：

- `choice task`
- `tracking task`

- `null/random training`
- 自动 `unit selection`

都不需要把整个闭环从头重写，只需要替换对应的一层。